Absolutely veere. 🔥

Hun assi **Phase 2** te aa gaye haan.

Main ik important decision explain karna chaunda haan pehlan.

📌 TRD Philosophy

Bohat saare developers TRD nu sirf tech stack da document bana dinde ne.

Main **eh galti nahi karna chaunda.**

Prism IQ da TRD ik **System Design Document** vi hovega.

Matlab is document nu padh ke kise developer nu bina kuch puchhe project implement karna aa jaana chahida.

Industry ch vi (Amazon, Microsoft, Atlassian) TRD da purpose eh hi hunda aa.

📄 Technical Requirements Document (TRD)

Project: Prism IQ

Version: 1.0

Status: Approved Architecture

Based on: PRD v1.0

Table of Contents

13. Performance Requirements
14. Error Handling
15. Logging & Monitoring
16. Deployment Strategy
17. Folder Structure
18. Future Scalability

---

1. Technical Overview

Prism IQ is designed as a modular analytics platform where each stage of the analytics lifecycle is implemented as an independent processing layer.

Rather than treating analytics as isolated features, the platform follows a structured pipeline:

```
Data Source
     ↓
Import
     ↓
Validation
     ↓
Profiling
     ↓
Cleaning
     ↓
EDA
     ↓
Machine Learning
     ↓
Business Intelligence
     ↓
Reports
```

Each stage has clearly defined responsibilities, ensuring maintainability, scalability, and easier testing.

---

2. Architecture Principles

The architecture follows these principles:

Modular

Each feature is isolated into dedicated modules.

Scalable

New import sources, ML algorithms, and reports can be added without modifying existing modules.

Stateless APIs

Backend APIs remain stateless, simplifying deployment and scaling.

Separation of Concerns

Frontend, backend, ML, and reporting responsibilities are strictly separated.

Pipeline-Based Processing

Every dataset passes through a deterministic processing pipeline.

---

3. Technology Stack

Frontend

- React 19
- TypeScript
- Vite
- Tailwind CSS
- shadcn/ui
- React Router
- TanStack Query
- Recharts
- Plotly

Backend

- FastAPI
- SQLAlchemy
- Pydantic
- Pandas
- NumPy
- Scikit-learn
- BeautifulSoup4
- HTTPX

- ReportLab

Database

PostgreSQL

Purpose:

- User metadata (future)
- Dataset metadata
- Analysis history
- Model metadata
- Report metadata

Development

- Git
- GitHub
- Docker (optional for future)
- Uvicorn

---

4. High-Level System Architecture

```
                User

                  |

            React Frontend

                  |

        FastAPI REST API Layer

                  |

  ____________________________________

   Import Service

   Cleaning Service
```

---

5. Data Processing Pipeline

Stage 1

Data Import

Supported:

- CSV
- Excel
- API
- HTML Tables

Output:

Validated DataFrame

Stage 2

Data Profiling

Generate:

- Missing Values
- Duplicates
- Statistics
- Cardinality
- Memory Usage
- Column Summary

Stage 3

Smart Cleaning

Automatically:

- Remove duplicates
- Handle missing values
- Convert data types
- Standardize categorical values
- Optional outlier handling

Output:

Cleaned Dataset

Stage 4

Exploratory Data Analysis

Generate:

- Distribution
- Correlation
- Trends
- KPIs
- Charts

Stage 5

Predictive Analytics

User selects:

Target Column

↓

System detects:

Regression

or

Classification

↓

Train multiple models

↓

Evaluate

↓

Best Model

↓

Prediction

Stage 6

Recommendation Engine

Generate:

- Business Insights
- Opportunities
- Risks
- Recommendations

- Executive Summary

Stage 7

Reporting

Generate:

- Executive PDF
- Cleaned CSV

---

6. Frontend Architecture

```
Pages

Landing

Import

Cleaning

Analytics

Prediction

Dashboard

Reports

Settings (Future)
```

Each page communicates with the backend through REST APIs.

Business logic remains on the backend.

---

7. Backend Architecture

The backend follows a layered architecture:

```
Routes

↓

Services
```

```
    ↓

    Repositories

    ↓

    Database
```

Responsibilities:

Routes

Receive HTTP requests.

Services

Business logic.

Repositories

Database interaction.

Models

ORM definitions.

Schemas

Validation.

Utilities

Shared helper functions.

---

8. Machine Learning Architecture

Workflow:

```
    Dataset

    ↓

    Feature Selection

    ↓

    Train/Test Split
```

```
      ↓

      Scaling (optional)

      ↓

      Multiple Models
```

Supported Models:

Regression:

- Linear Regression
- Random Forest Regressor
- Decision Tree Regressor

Classification:

- Logistic Regression
- Decision Tree
- Random Forest

Future:

XGBoost

LightGBM

CatBoost

---

9. Database Architecture

PostgreSQL stores:

- Dataset metadata
- Processing history
- Generated reports
- Model metadata

Actual uploaded files remain in the filesystem.

This minimizes database size while maintaining metadata consistency.

---

10. File Storage Strategy

```
      uploads/
```

```
raw/

cleaned/

reports/

models/

temp/
```

Every uploaded dataset receives a unique Dataset ID.

All generated artifacts reference this ID.

---

11. API Standards

RESTful design.

Example:

```
POST /upload

POST /profile

POST /clean

POST /eda

POST /train

POST /predict

GET /dashboard

GET /report
```

JSON request/response format.

Consistent HTTP status codes.

---

12. Security Strategy

Input validation.

File type validation.

Maximum upload size.

Safe filename generation.

Parameterized SQL.

CORS configuration.

Error sanitization.

---

13. Performance Requirements

Support:

~100,000 rows.

EDA charts should avoid rendering millions of points directly; sampling strategies may be applied when appropriate.

Heavy computations should be isolated into dedicated service modules.

---

14. Error Handling

Every service returns standardized responses.

```
Success

Status

Message

Data

Errors
```

Validation errors never crash the application.

---

15. Logging Strategy

Application Logs

API Logs

Processing Logs

ML Training Logs

Report Generation Logs

Future:

Structured JSON logging.

---

16. Deployment Strategy

Frontend:

Vercel

Backend:

Render

Database:

Neon PostgreSQL

Environment variables:

```
DATABASE_URL

API_KEY

UPLOAD_DIR

SECRET_KEY
```

---

17. Recommended Folder Structure

```
prism-iq/

|

├── frontend/

|

├── backend/

|

├── uploads/
```

```
      |
      ├── reports/
```

Detailed backend and frontend structures will be defined in the implementation documents.

---

18. Scalability Roadmap

Future architecture supports:

- Authentication
- User projects
- Team workspaces
- Cloud storage
- Database connectors
- Time-series forecasting
- Explainable AI
- Plugin system
- Background workers (Celery + Redis)
- Containerized deployment (Docker)

✅ Architecture Decisions (Frozen)

Area	Decision
Frontend	React + Vite + TypeScript
Styling	Tailwind CSS + shadcn/ui
Backend	FastAPI

ML	Scikit-learn
Database	PostgreSQL (Neon)
ORM	SQLAlchemy
Charts	Plotly + Recharts
Reports	ReportLab
File Storage	Local filesystem (MVP)
API Style	REST

📌 Before Phase 3 (Feature-wise Implementation Plan)

Veere, TRD v1.0 is solid, but **main isnu hor enterprise-level banauna chaunda haan.**

Feature plan likhan ton pehlan, main **Backend Schema (ERD + Tables + Relationships)** design karna pasand karaanga. Reason simple aa:

> **Features should be built around the database, not the other way around.**

Je schema pehlan freeze ho gaya, taan:

- API design cleaner hovegi.
- Backend services stable rahange.
- Feature implementation plan automatically precise ban ju.
- Database migrations dubara-dubara change nahi karniyan paingiyan.

So meri recommendation:

Order slightly change kariye:

1. ✅ PRD

2. ✅ TRD
3. **Backend Schema (ERD & DB Design)**
4. Feature-wise Implementation Plan
5. Project Flow
6. UI/UX Discussion
7. UI/UX Docs

Eh order real-world product development de bahut kareeb aa, te main strongly recommend karaanga ki assi eh follow kariye. 🚀